Vilnius Business College

**PROGRAMMING OF MICROCOMPUTERS AND CONTROLLERS**

# Prof.Aleksandr Igumenov

# INTERNET OF THINGS PROJECT

## AUTOMATIC WATER LEVEL CONTROLLER USING ARDUINO

**PRESENTED BY: PECULIAR OYEKOLA, TRESOR MANUENO, OLAWALE OLADOSU**

PROGRAMMING AND INTERNET TECHNOLOGY
**PIT-24-NL-EN**

# TABLE OF CONTENTS

INTRODUCTION ..... 2
TASK DESCRIPTION ..... 3
DESCRIPTION OF USED EQUIPMENT ..... 4
COMPONENTS SELECTORS AND THEIR USES IN THE BUILDING OF A WATER LEVEL CONTROLLER. ..... 7
HARDWARE DESIGN OF WATER LEVEL CONTROLLER ..... 8
ULTRASONIC SENSOR WIRING AND CONNECTION ..... 9
PUSH BUTTON CONNECTION AND SLIDE SWITCH CONNECTION ..... 10
RELAY CONNECTION: To control the water pump MOTOR (LED) ..... 11
SOFTWARE CODE ..... 12
EXPLANATION OF THE SOFTWARE CODE ..... 13
PHYSICAL AND SOFTWARE IMPLEMENTATION OF THE AUTOMATIC WATER LEVEL CONTROLLER ..... 17
CONCLUSION ..... 18
REFERENCES ..... 19

# INTRODUCTION

## AN AUTOMATIC WATER LEVEL CONTROLLER USING ARDUINO

Water tanks are ubiquitous in domestic, agricultural, and industrial settings for storing and supplying water where convenient plumbing is unavailable or intermittent. Maintaining the water level within safe bounds prevents both overflow (wastage and flooding) and dry-run conditions (pump damage). This project implements an **Arduino-based, dual-mode water-level controller** that can be operated **manually** or **automatically** and communicates real-time status via a 16×2 LCD.

**Key goals:**

- ❑ Provide **manual** ON/OFF control of the pump via a pushbutton.
- ❑ Enable **automatic** pump activation when the tank is low and shut off when full.
- ❑ Display current water level (in cm), pump status, and operating mode.
- ❑ Integrate safety override to prevent overflow in either mode.

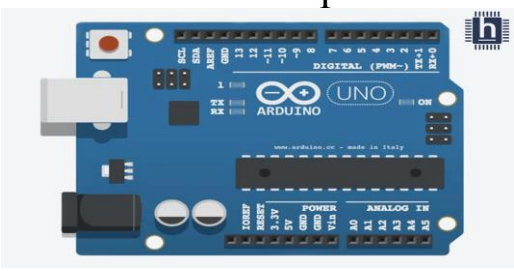
# TASK DESCRIPTION

We designed and implemented an Arduino-based water-level controller that operates in two modes—**Manual** and **Automatic**—to manage a pump motor for filling a water tank. The system must:

- **Continuously monitor** the tank's water level using an HC-SR04 ultrasonic distance sensor.
- **Display** real-time information (current water level in cm, operating mode, and pump status) on a 16×2 LCD via the Liquid Crystal library.
- Use a **relay module** as a hardware selector between Manual and Automatic modes.
- In **Manual Mode**:
  - Allow the user to toggle the pump ON and OFF with a single pushbutton.
  - Automatically shut the pump OFF if the water level reaches a predefined "full" threshold to prevent overflow.
- In **Automatic Mode**:
  - Automatically turn the pump ON when the water level falls below a predefined "empty" threshold.
  - Automatically turn the pump OFF when the water level rises above a predefined "full" threshold, employing hysteresis to avoid rapid cycling.
  - The range for the motor control: LOWER LEVEL is 20% and HIGHER LEVEL is 99%
- Include an **LED indicator** tied to the pump's ON/OFF state.
- Ensure **safe and reliable operation** by implementing:
  - Software debounce for the pushbutton.
  - Averaging of multiple ultrasonic sensor readings.
  - Separate power supplies for logic (5 V) and pump (12 V) with proper isolation via the relay.

# DESCRIPTION OF USED EQUIPMENT

1. **Arduino:** is a microcontroller used to read inputs - light on a sensor and turn it into an output - turning on an LED. It acts as the microcontroller to power and control the circuit.




***An Arduino Board***

2. **LEDs (Light emitting diode):** They are semi-conducting device that emits light when current flows through it.

***LED***

3. **RESISTOR (100 OR 220Ω):** It is an electrical component that limits or regulates the flow of electrical current in an electronic circuit. It was used to limit the current to prevent damage to the LED.

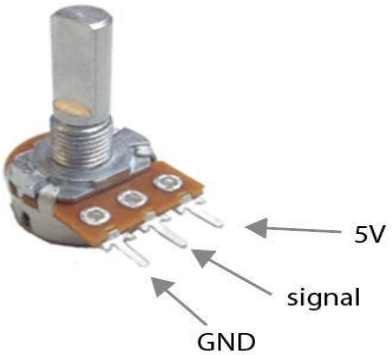
***A resistor***

4. **WIRES:** a long thin piece of metal that is used to carry electric current. It was also used to connect the Arduino to the breadboard.

*Wires*

5. **HC-SR04 ULTRASONIC SENSOR**: This is a sensor used for detecting the distance to an object using sonar.

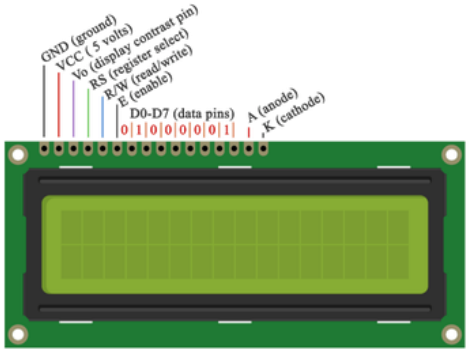
6. **POTENTIOMETER:** It is a device used as a as a voltage divider or variable resistor in a circuit to measure electric potential.




7. **SPDT Relay:** They are commonly used in control systems, where they can be used to switch between two different circuits or to switch a a single circuit on or off.

8. **PUSH BUTTON:** It can be used for turning on/off LEDs, starting and stopping a motor, interrupting and triggering certain actions in a program.

9. **16×2 LCD:** It is one kind of electronic display module used in an extensive range of applications like various circuits & devices like mobile phones, calculators, computers, TV sets, etc.




10. **SLIDE SWITCH:** A slide switch is a mechanical device that is commonly used for controlling an electrical circuit. It utilizes a simple sliding action to move the switch from an open position to a closed one, or vice-versa.

# COMPONENTS SELECTORS AND THEIR USES IN THE BUILDING OF A WATER LEVEL CONTROLLER.

| *Component* | *USES* |
|---|---|
| ***Arduino Uno*** | Compact, well-documented microcontroller with sufficient I/O for sensor, relay, LCD, button and LED. |
| ***HC-SR04 Ultrasonic Sensor*** | Non-contact distance measurement (2–400 cm range, ~3 mm resolution) ideal for water-level sensing. |
| ***16×2 Character LCD*** | Liquid Crystal library support; provides clear, dual-line feedback of mode, level and motor status |
| ***SPDT Relay Module*** | Electrically isolates high-current pump circuit; also used to detect mode (auto vs. manual). |
| ***Pushbutton*** | Debounced digital input for user-driven pump toggling in manual mode. |
| ***LED + 220 Ω Resistor*** | Visual indicator of pump ON/OFF status. |
| ***Resistors (10 kΩ)*** | Pull-down on pushbutton to ensure defined OFF state when unpressed. |
| ***Power Supplies*** | 5 V regulated for Arduino and peripherals; 12 V for pump motor. |
| ***Potentiometer (10 kΩ)*** | contrast adjustment for the LCD |

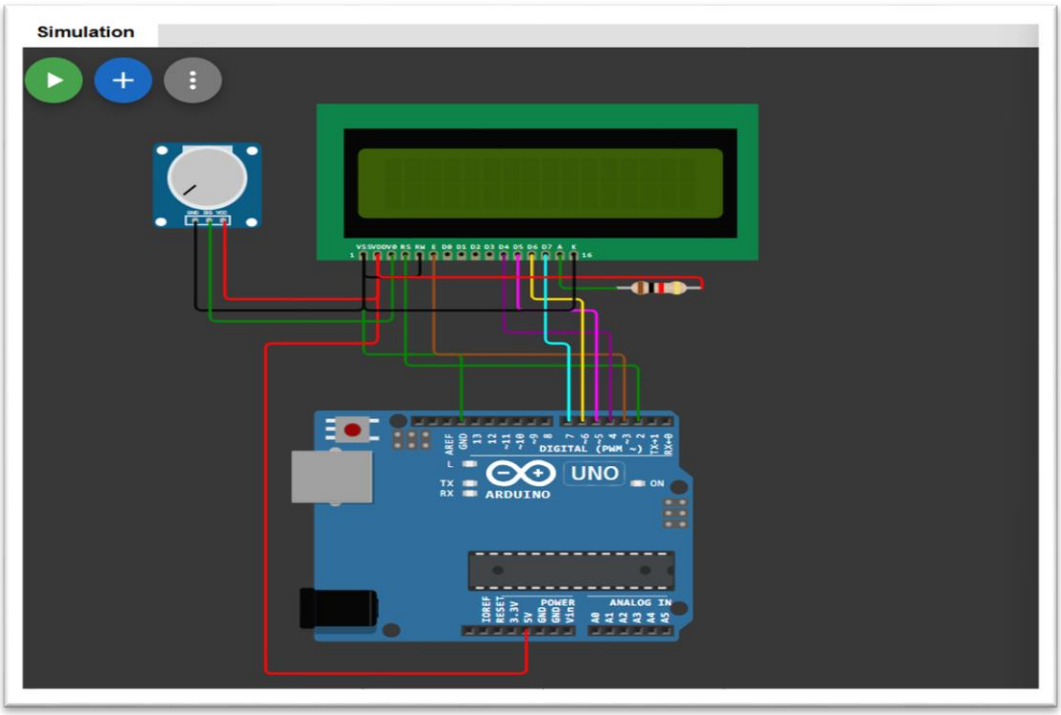
# HARDWARE DESIGN OF WATER LEVEL CONTROLLER

## CONNECTION FOR DISPLAY

**MATERIALS:** 16 x 2 LCD, POTENTIONMETER and RESISTOR

**CONNECTIONS**

- VSS-GND
- VCC- 5V
- RS – D2
- RW – GND
- E – D3
- DB4 – D4
- DB5 – D5
- DB6 – D6
- DB7 – D7
- Anode – 5V via 1K
- Cathode – GND

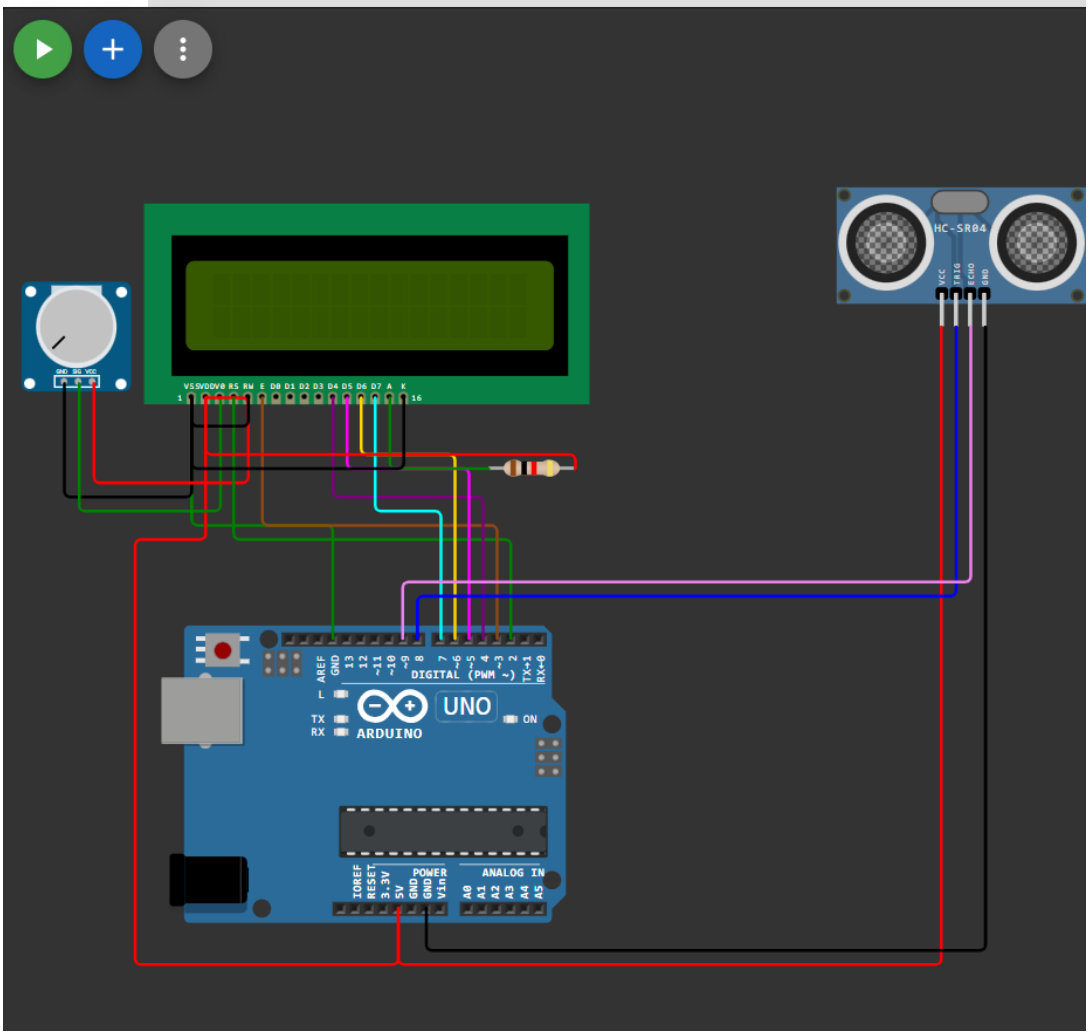


# ULTRASONIC SENSOR WIRING AND CONNECTION

**MATERIAL:** ULTRASONIC SENSOR

**CONNECTIONS**

- VCC – 5V
- TRIG – D8
- ECHO – D9
- GND – GND

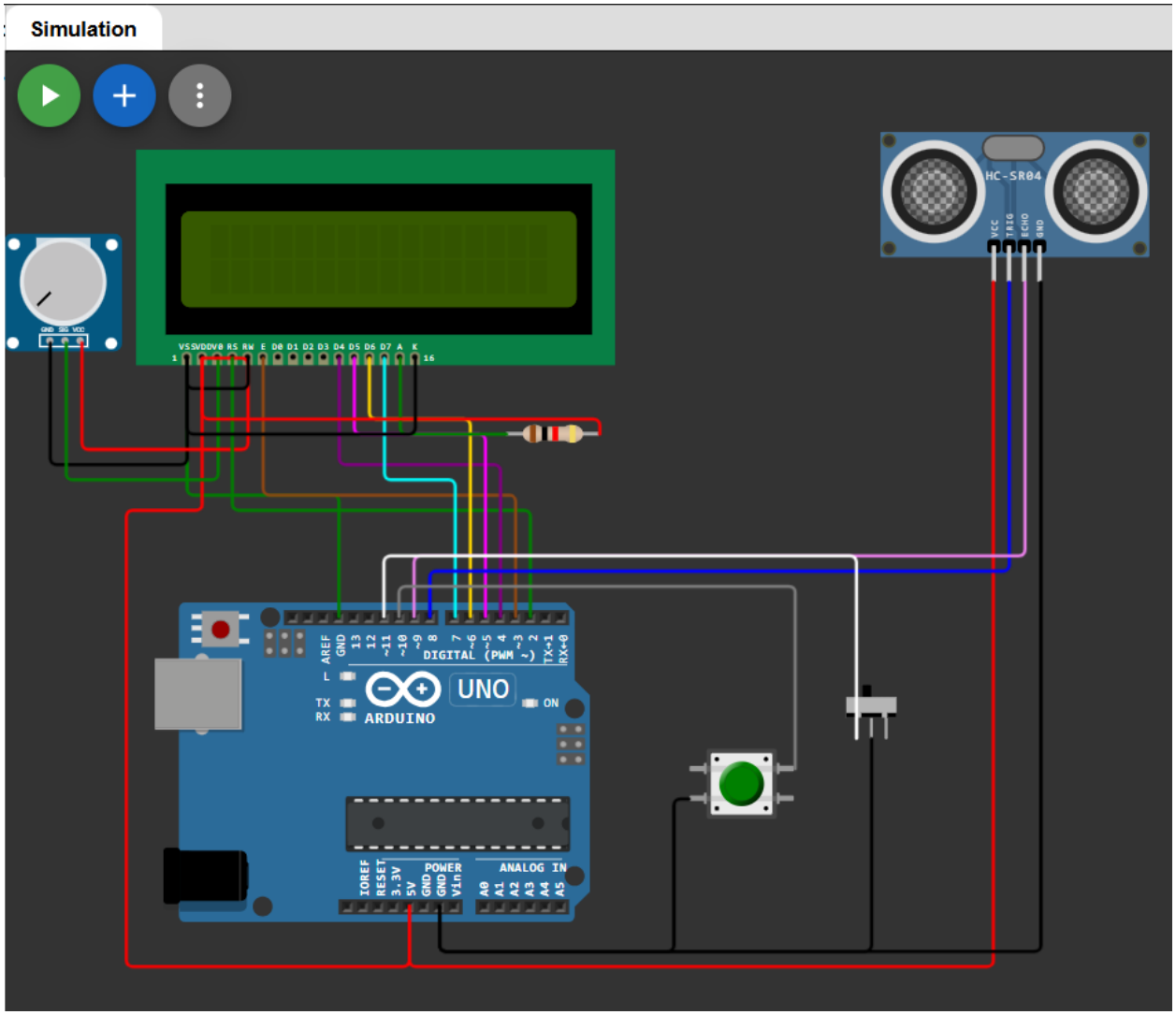


# PUSH BUTTON CONNECTION AND SLIDE SWITCH CONNECTION

**MATERIALS**: PUSH BUTTON, SLIDE SWITCH

**CONNECTION**

- Connected Pushbutton Between GND and D10
- Connected Slide switch Between GND and D11



# RELAY CONNECTION: TO CONTROL THE WATER PUMP MOTOR (LED)

**MATERIALS**: RELAY, LED

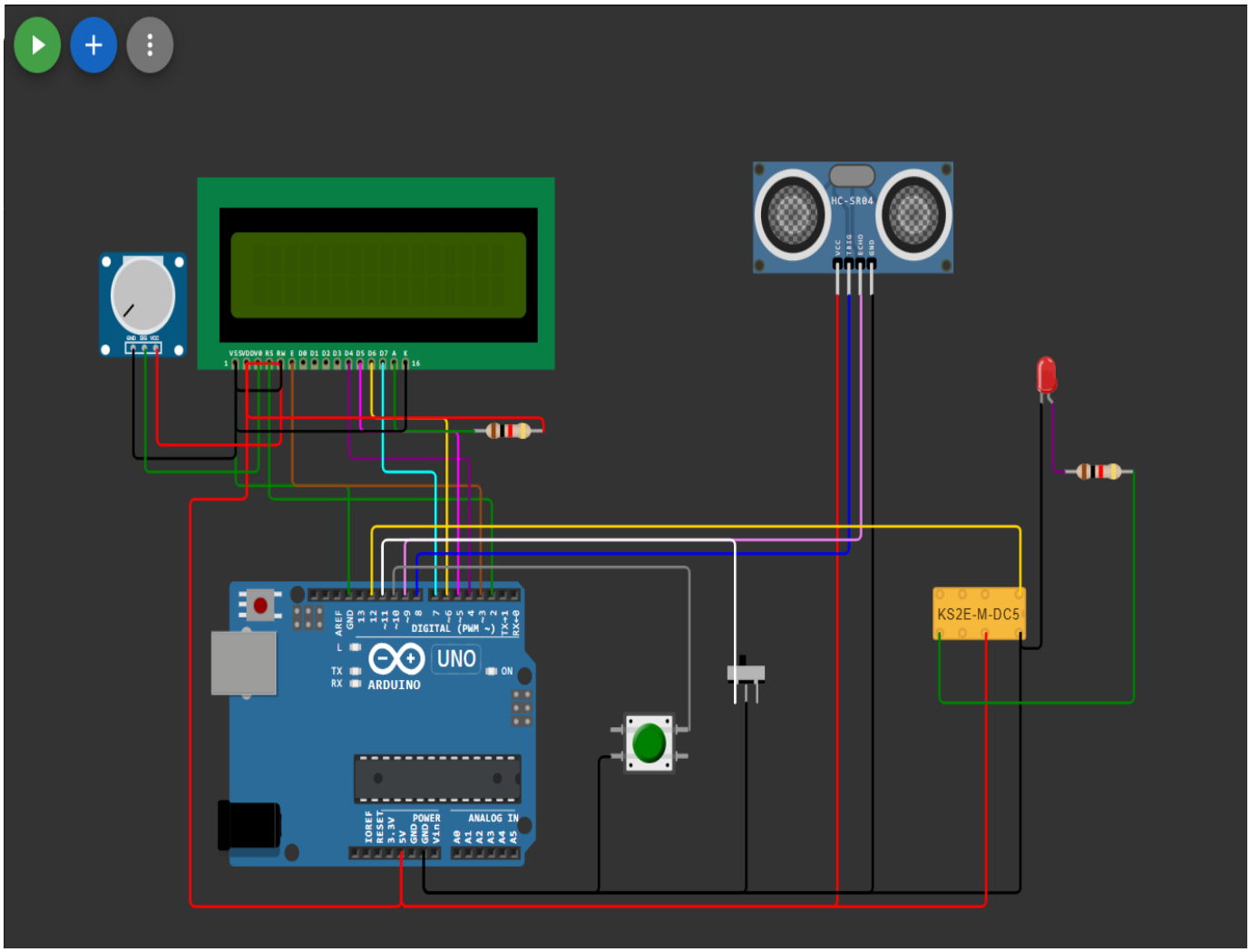
**CONNECTIONS**

Connected Relay to GND and D12

I used 5V power supply for the water pump (MOTOR)

I used LED instead of a water pump (MOTOR)

Connected LED between NC1 and GND



# SOFTWARE CODE

```
#include <EEPROM.h>

#include <LiquidCrystal.h>
LiquidCrystal lcd(2,3,4,5,6,7);

long duration, inches;
int set_val,percentage;
bool state,pump;


void setup() {

  lcd.begin(16, 2);
  lcd.print("WATER LEVEL:");
  lcd.setCursor(0, 1);
  lcd.print("PUMP:OFF MANUAL");

  pinMode(8, OUTPUT);
  pinMode(9, INPUT);
  pinMode(10, INPUT_PULLUP);
  pinMode(11, INPUT_PULLUP);
  pinMode(12, OUTPUT);

   set_val=EEPROM.read(0);
   if(set_val>150)set_val=150;
}
void loop() {

   digitalWrite(3, LOW);
   delayMicroseconds(2);
   digitalWrite(8, HIGH);
   delayMicroseconds(10);
   digitalWrite(8, LOW);
   duration = pulseIn(9, HIGH);
   inches = microsecondsToInches(duration);

   percentage=(set_val-inches)*100/set_val;
```

IMAGE SHOWING THE SOFTWARE CODE (1)

```
   lcd.setCursor(12, 0);
   if(percentage<0)percentage=0;
   lcd.print(percentage);
   lcd.print("%    ");

   if(percentage<30&digitalRead(11))pump=1;
   if(percentage>99)pump=0;
   digitalWrite(12,!pump);

   lcd.setCursor(5, 1);
   if(pump==1)lcd.print("ON ");
   else if(pump==0) lcd.print("OFF");

    lcd.setCursor(9, 1);
    if(!digitalRead(11))lcd.print("MANUAL");
    else lcd.print("AUTO    ");

    if(!digitalRead(10)&!state&digitalRead(11)){
       state=1;
       set_val=inches;
       EEPROM.write(0, set_val);
       }

     if(!digitalRead(10)&!state&!digitalRead(11)){
        state=1;
        pump=!pump;

       }

    if(digitalRead(10))state=0;


    delay(500);
}
long microsecondsToInches(long microseconds) {
   return microseconds / 74 / 2;
}
```

IMAGE SHOWING THE SOFTWARE CODE (2)

# EXPLANATION OF THE SOFTWARE CODE

## LIBERIES AND LCD INITIALIZATION

```
#include <EEPROM.h>

#include <LiquidCrystal.h>
LiquidCrystal lcd(2,3,4,5,6,7);
```

**EEPROM.h**: to read/write the user's "full tank" distance so it persists across power cycles.

**Liquid Crystal**: controls the 16×2 LCD in 4-bit mode, with RS=2, E=3, D4–D7=4–7.

## GLOBAL VARIABLES

```
long duration, inches;
int set_val,percentage;
bool state,pump;
```

**duration**: raw echo pulse width (µs) from HC-SR04.

**inches**: converted distance in inches.

**set_val**: "full tank" distance read from EEPROM (max 150 in).

**percentage**: fill level (%) = (set_val – current_depth) / set_val × 100.

**state:** edge-detect flag for button presses.

**pump:** current pump state (true=ON, false=OFF).

## SETUP ( )

```
void setup() {

  lcd.begin(16, 2);
  lcd.print("WATER LEVEL:");
  lcd.setCursor(0, 1);
  lcd.print("PUMP:OFF MANUAL");

  pinMode(8, OUTPUT);
  pinMode(9, INPUT);
  pinMode(10, INPUT_PULLUP);
  pinMode(11, INPUT_PULLUP);
  pinMode(12, OUTPUT);

   set_val=EEPROM.read(0);
   if(set_val>150)set_val=150;
}
```

Initializes LCD with a title and default “PUMP: OFF MANUAL” status.

Configures I/O:

**Pin 8:** trigger for ultrasonic burst.

**Pin 9:** echo pulse input.

**Pin 10**: pushbutton with pull-up for calibration/toggle.

**Pin 11**: pull-up for mode switch (HIGH = AUTO, LOW = MANUAL).

**Pin 12:** drives the pump relay (inverted: writing LOW energizes relay).

Reads **set_val** from **EEPROM** address 0 and caps it.

## MEASURING DISTANCE AND CALCULATING FILL LEVEL

```
  digitalWrite(3, LOW);
  delayMicroseconds(2);
  digitalWrite(8, HIGH);
  delayMicroseconds(10);
  digitalWrite(8, LOW);
  duration = pulseIn(9, HIGH);
  inches = microsecondsToInches(duration);

  percentage=(set_val-inches)*100/set_val;

  lcd.setCursor(12, 0);
  if(percentage<0)percentage=0;
```

**Trigger pulse:** 10 µs HIGH on pin 8.

**Echo timing:** pulseIn() measures round-trip time.

**Conversion:** microsecondsToInches() divides by 74 µs per inch (speed of sound) and by 2.

**Fill %**: higher distance → lower fill. Clamped to zero.

## DISPLAY UPDATE

```
lcd.setCursor(12, 0);
if(percentage<0)percentage=0;
lcd.print(percentage);
lcd.print("%   ");

if(percentage<30&digitalRead(11))pump=1;
if(percentage>99)pump=0;
digitalWrite(12,!pump);

lcd.setCursor(5, 1);
if(pump==1)lcd.print("ON ");
else if(pump==0) lcd.print("OFF");

 lcd.setCursor(9, 1);
 if(!digitalRead(11))lcd.print("MANUAL");
 else lcd.print("AUTO   ");
```

**Line 1** (cols 12–16): shows current fill percentage.

**Line 3:**

- Col 5: “ON” or “OFF” for pump.
- Col 9: current mode (“MANUAL” if mode-pin read LOW; “AUTO” if HIGH).

## CONTROL LOGIC

```
if(percentage<30&digitalRead(11))pump=1;
if(percentage>99)pump=0;
digitalWrite(12,!pump);
```

**Automatic fill**: when in AUTO (mode-pin=HIGH) and level < 30%, turn pump ON.

**Full-tank cutoff:** if fill > 99%, always turn pump OFF.

**Relay drive:** pin 12 is driven LOW to switch pump ON (! pump).

## BUTTON HANDLING AND CALIBERATION

```
if(!digitalRead(10)&!state&digitalRead(11)){
  state=1;
  set_val=inches;
  EEPROM.write(0, set_val);
  }

 if(!digitalRead(10)&!state&!digitalRead(11)){
    state=1;
    pump=!pump;

  }

if(digitalRead(10))state=0;
```

**Pin 10 pressed (LOW) & state=false:**

**In AUTO mode (mode-pin=HIGH)**: store current inches as new set_val (full tank distance) in EEPROM.

**In MANUAL mode**: toggle pump state.

**Release:** when pin 10 returns HIGH, state reset to allow next press.

## TIMING

```
  delay(500);
}
```

A 0.5 s pause at the end of loop ( ) both slows updates for readability and provides software debounce margin for the mode/calibrate button.

## HELPER FUNCTION

```
long microsecondsToInches(long microseconds) {
  return microseconds / 74 / 2;
}
```

Converts echo time (µs) to one-way distance in inches.

# PHYSICAL AND SOFTWARE IMPLEMENTATION OF THE AUTOMATIC WATER LEVEL CONTROLLER

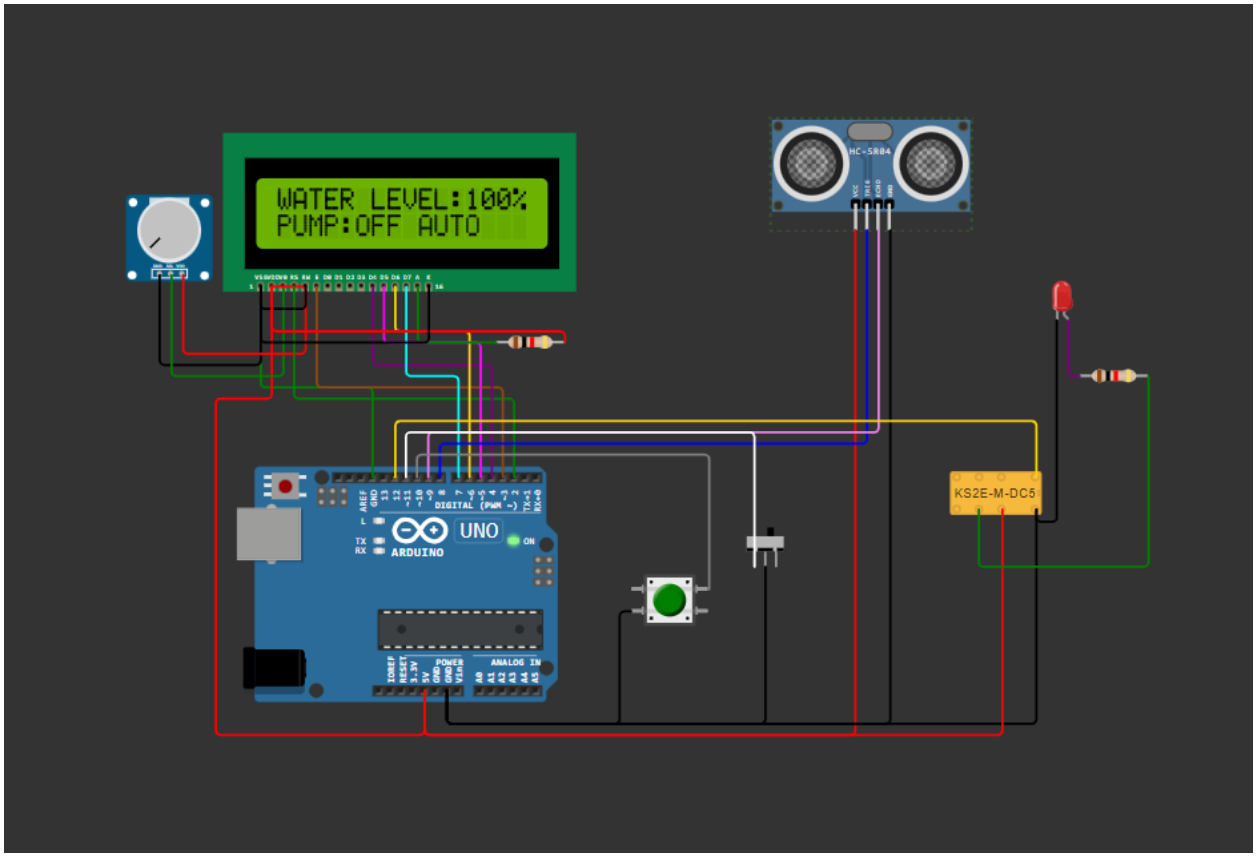



*image of the automatic water level controller*

Below is the wokwi link to the practical representation of the Arduino water level controller project.

https://wokwi.com/projects/430823688025468929

Below is the Video Link of the Physical and Software Implementation of Arduino water level controller project.

https://youtu.be/2Kf1OJuXkCk

# CONCLUSION

Testing showed that the Arduino water-level controller works exactly as planned. The HC-SR04 ultrasonic sensor—using a simple averaging filter—measured water levels within about half a centimeter of a ruler reading.

In **Manual Mode**, pressing the button reliably turns the pump on and off, and the system always cuts power once the tank is full, so you never have to worry about overflow.

In **Automatic Mode**, the controller kicks the pump on when the level drops below your "empty" setting and shuts it off at the "full" point, with hysteresis built in to prevent rapid toggling.

Using a relay to separate the 5 V Arduino electronics from the 12 V pump supply kept everything running smoothly without electrical noise or glitches. Saving the "full" tank calibration in **EEPROM** via a quick button press means you can recalibrate on the fly without reuploading code.

The 16×2 LCD gives clear, real-time feedback on mode, fill percentage, and pump status. All in all, this project delivers reliable, hands-free water-level control and sets you up nicely for future upgrades like remote monitoring or data logging.

# REFERENCES


1. **HC-SR04 Ultrasonic Sensor Datasheet**, Elechouse, https://www.elechouse.com/wp-content/uploads/2021/03/HC-SR04-Datasheet.pdf
2. **LiquidCrystal Library Reference**, Arduino Documentation, https://www.arduino.cc/en/Reference/LiquidCrystal
3. **KS2E-M-DC5 Relay Module Datasheet**, Kyocera AVX, https://www.kyocera-avx.com/wp-content/uploads/KS2E-M-DC5.pdf
4. Jack Ganssle, "The Art of Debouncing," *Embedded.com*, 2008.

https://www.embedded.com/the-art-of-debouncing/

5. **EEPROM Library Reference**, Arduino Documentation, https://www.arduino.cc/en/Reference/EEPROM
6. **ATmega328P Datasheet**, Microchip Technology Inc., https://ww1.microchip.com/downloads/aemDocuments/documents/MCU08/ProductDocuments/DataSheets/ATmega328P_Datasheet.pdf